Hostel Hive

Performance Test Results

By

Hasinthaka Piyumal

SE/2021/036

Navon Sanjuni

SE/2021/061

Thagya Kavindi

SE/2021/062

Tasheen Darshika

SE/2021/030

A report submitted in partial fulfilment of the requirements for the degree of Bachelor of Science Honors in Software Engineering (B.Sc.SE)

SENG 34213: System Development Project

Software Engineering Teaching Unit

Faculty of Science

University of Kelaniya

Sri Lanka

2026

Version No	Date	Author(s)	Summary of Changes
1.0	2026/10/26	Hasinthaka, Sanjuni, Thagya, Tasheen	First version of the performance test results.

Table 1: Version History

Table of Contents

1 Introduction 1

1.1 Purpose 1

1.2 What the SRS Asks For 1

1.3 Words We Use 2

2 How We Tested 3

2.1 Tools 3

2.2 Backend Test Setup 3

2.3 Test Data 3

2.4 What One User Does 4

2.5 The Tests 5

3 Summary 6

4 Backend Results 8

4.1 Load Test 8

4.2 100 Users at the Same Time (NFR002) 9

4.3 Curfew Rush 10

4.4 Spike Test 11

4.5 Stress Test 12

4.6 Reports on Five Years of Data (NFR010) 14

4.7 Server Resources 16

5 Live API on Render 18

6 Web Dashboard Results 20

7 Conclusion 25

8 References 26

List of Tables

Table 1: Version History II

Table 2: SRS Performance Requirements 1
Table 3: Words Used in This Report 2

Table 4: Backend Test Setup 3
Table 5: Test Data 4
Table 6: User Mix 4
Table 7: Tests We Ran 5

Table 8: Backend Results (Req/s is the average over the whole test) 6
Table 9: Web Dashboard Lighthouse Scores (out of 100) 6
Table 10: SRS Requirements Result 7

Table 11: Peak CPU and Memory (100% means one full core) 17

Table 12: Render Response Times 18

Table 13: Web Dashboard Load Times 20

List of Figures

Figure 1: k6 dashboard for the load test 8
Figure 2: k6 result for 100 users. Every threshold and check passed 9
Figure 3: k6 dashboard for 100 users 10
Figure 4: k6 dashboard for the curfew rush 11
Figure 5: k6 dashboard for the spike test 12
Figure 6: Stress test minute by minute 13
Figure 7: Response time as traffic goes up in the stress test 13
Figure 8: k6 dashboard for the stress test 14
Figure 9: Time for each report type with 5 wardens 15
Figure 10: k6 dashboard for the reports test 16

Figure 11: k6 result for the light check on Render 18
Figure 12: k6 dashboard for the Render check 19

Figure 13: Lighthouse, student login on desktop 21

Figure 14: Lighthouse, staff login on desktop 22
Figure 15: Lighthouse, student login on mobile 23
Figure 16: Lighthouse, staff login on mobile 24

CHAPTER 1

INTRODUCTION

1.1 Purpose

This document is Deliverable 9 (*Performance Test Results*) in Section 10.1 of the SENG 34213 project guideline. The guideline puts performance testing in Sprint 4 (Testing and Hardening), and the final report needs a summary of these results.

We wanted to answer three simple questions. Is HostelHive fast enough for normal use? Does it stay fast when many people use it at the same time? And how does it behave after five years of attendance records? We tested both the backend API and the web dashboard.

1.2 What the SRS Asks For

SRS ID	**Requirement**	**How we checked it**
NFR001	95% of requests are answered within 2 seconds under normal load.	Every load test fails if the 95th percentile (p95) goes over 2 seconds.
NFR002	At least 100 users at the same time without slowing down.	100 users used the system together for 10 minutes.
NFR010	The database keeps 5 years of attendance logs without manual archiving.	We filled the database with 5 years of scans and ran the tests and reports on it.

Table 2: SRS Performance Requirements

1.3 Words We Use

Word	Meaning
Virtual user	A script that acts like one person using the app.
Median	Half the requests were faster than this.
p95	95% of the requests were faster than this. This is the number NFR001 talks about.
Requests per second	How many API calls the server handled each second.
Think time	A short pause between actions, like a real person reading the screen.
FCP	First Contentful Paint. When the first text or image appears.
LCP	Largest Contentful Paint. When the main part of the page appears.
TBT	Total Blocking Time. How long the page could not react to clicks while loading.
CLS	Cumulative Layout Shift. How much the page jumps around while loading. Lower is better.

Table 3: Words Used in This Report

CHAPTER 2

HOW WE TESTED

2.1 Tools

- **k6** (version 2.3.0), an open source load testing tool from Grafana, for the backend API. Each test describes what the users do, and k6 measures every request.
- **Lighthouse** (version 13.5.0) from Google, for the web dashboard. It loads a page in Chrome and scores how fast it is.

2.2 Backend Test Setup

The backend load tests ran against a copy of our backend in Docker. The live server on Render is shared and on a small plan, so we gave it a light check instead (Chapter 5).

Part	**Setup**
Backend	Our Go backend from the `develop` branch, built with Go 1.27.3, limited to 1 CPU core and 512 MB of memory.
Database	PostgreSQL 16 with all our migrations applied.
Settings	`APP_ENV=production`, so logging, error handling and timeouts are the same as on the real server.
Login	Test accounts with test login tokens. Account loading, role checks and all handlers are the normal application code.

Table 4: Backend Test Setup

2.3 Test Data

An empty database is always fast, so we filled it first with fake data that looks like a real hostel.

Data	Amount
Students	1,000, each with a guardian, a QR code and a bed
Rooms	4 blocks, 63 rooms per block, 4 beds per room
Staff	1 admin, 2 wardens, 2 sub wardens, 3 security staff
Gate scanners	2 for going in, 2 for going out
Attendance scans	About 2.5 million, covering 5 years
Leave records	30 per student over 5 years
Complaints	5 per student

Table 5: Test Data

2.4 What One User Does

Most backend tests use the same mix of actions. Each virtual user picks one of these, does it, waits 1 to 3 seconds, and picks again.

Share	Who	What they do
60%	A student opens the app	Loads their account, profile, attendance history, passes, unread count and announcements
20%	A warden checks the dashboard	Loads the attendance summary, who is in or out, the student list, pending leaves and pending complaints
10%	A student sends requests	Sends a late access request and a complaint
10%	A gate scanner	Sends one QR scan

Table 6: User Mix

2.5 The Tests

Test	What happens
Load	A normal busy evening. Up to 50 users for 5 minutes.
100 users	100 users together for 10 minutes (NFR002).
Curfew rush	Gate scans only, rising to 60 scans per second.
Spike	300 users arrive within 20 seconds.
Stress	No think time. Traffic keeps going up to find how much the server can take.
Reports	5 wardens run reports and exports on 5 years of data (NFR010).
Render check	5 users for 1 minute on the live API.
Web dashboard	Lighthouse on the student and staff login pages, on mobile and desktop.

Table 7: Tests We Ran

CHAPTER 3

SUMMARY

Test	Users	Req/s	Median	p95	Result
Load	50	98	6 ms	20 ms	**Pass**
100 users	100	226	6 ms	19 ms	**Pass**
Curfew rush	gates	26	10 ms	21 ms	**Pass**
Spike	300	250	38 ms	152 ms	**Pass**
Stress	up to 600	346	12 ms	1,027 ms	**Pass**
Reports	5 wardens	1	219 ms	see 4.6	See 4.6
Render check	5	8	80 ms	97 ms	**Pass**

Table 8: Backend Results (Req/s is the average over the whole test)

Page	Performance	Accessibility	Best practices	SEO
Student login, desktop	100	96	100	91
Staff login, desktop	100	96	100	91
Student login, mobile	86	96	100	91
Staff login, mobile	88	96	100	91

Table 9: Web Dashboard Lighthouse Scores (out of 100)

SRS ID	Result	Why
NFR001	**Pass**	Under normal load the p95 was around 20 ms, far inside 2 seconds.
NFR002	**Pass**	With 100 users the p95 stayed at 19 ms, the same as with 50 users.
NFR010	**Pass**	5 years of scans fit with no archiving, and all the tests above ran on that data.

Table 10: SRS Requirements Result

In short, the backend stays fast for students, wardens and the gates, even at several times the size of a real hostel, and the web dashboard loads in about half a second on desktop.

CHAPTER 4

BACKEND RESULTS

4.1 Load Test

Up to 50 users used the system for 8 minutes in total. The server answered about 47,000 requests. The median was 6 ms and the p95 was 20 ms.

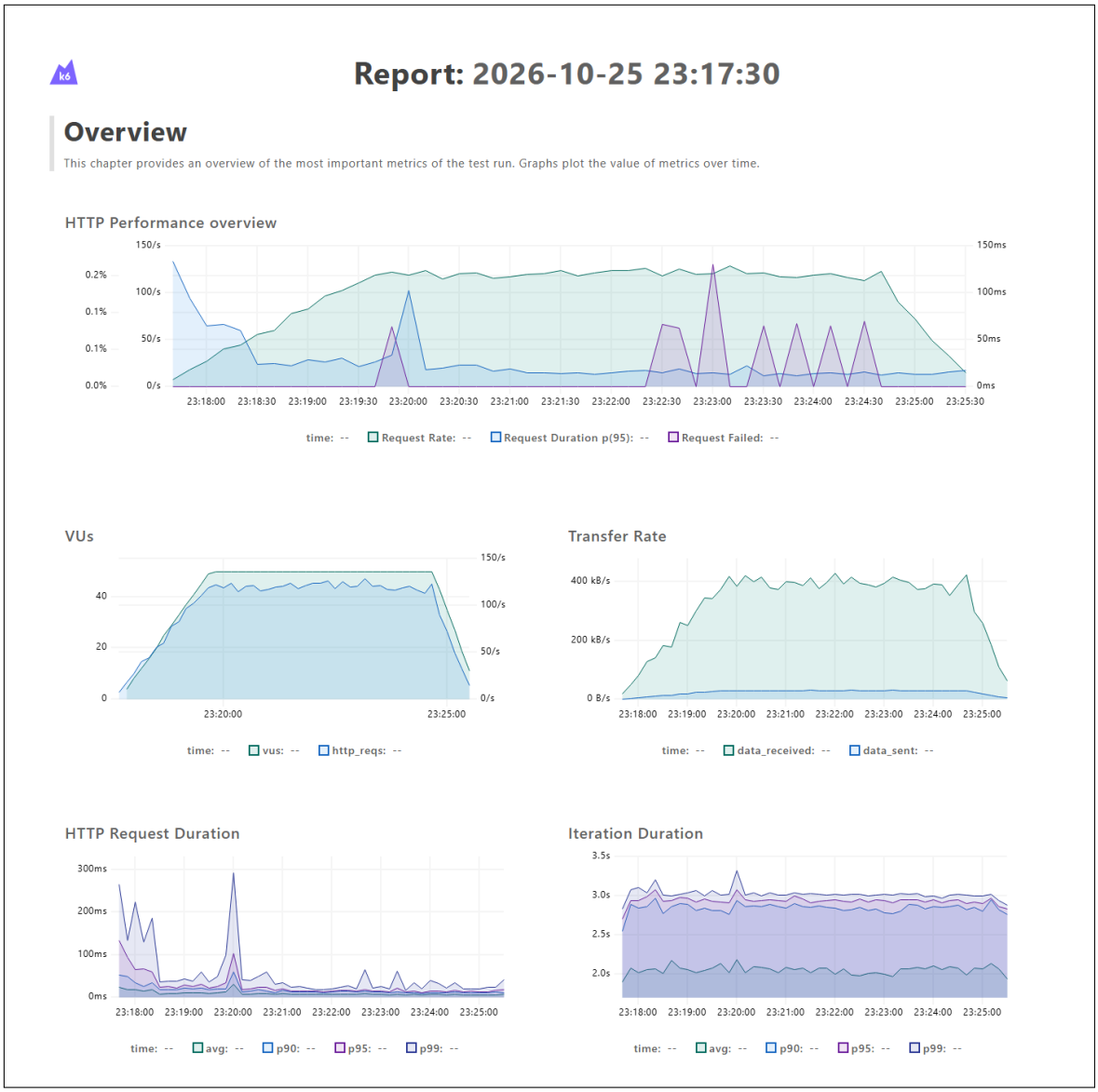


Figure 1: k6 dashboard for the load test

A few requests came back as `409 Conflict`. This is a correct answer: the random student already had a late access request for that day. Our checks accept it, which is why all checks

passed.

Result: **Pass**.

4.2 100 Users at the Same Time (NFR002)

100 users used the system together for 10 minutes. The server answered about 156,000 requests, around 226 per second. The p95 was 19 ms, which is no slower than with 50 users, so there was no slow down.

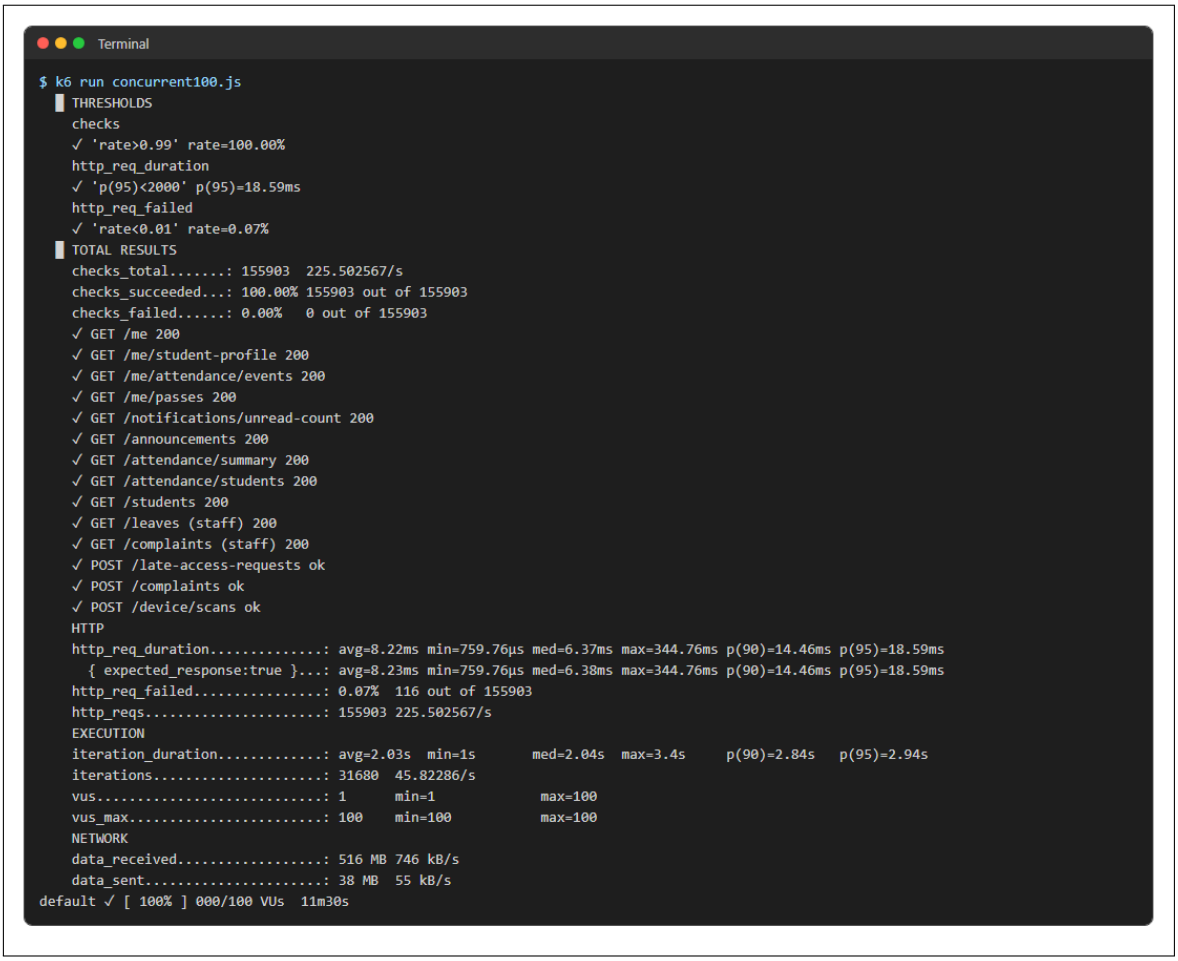


Figure 2: k6 result for 100 users. Every threshold and check passed

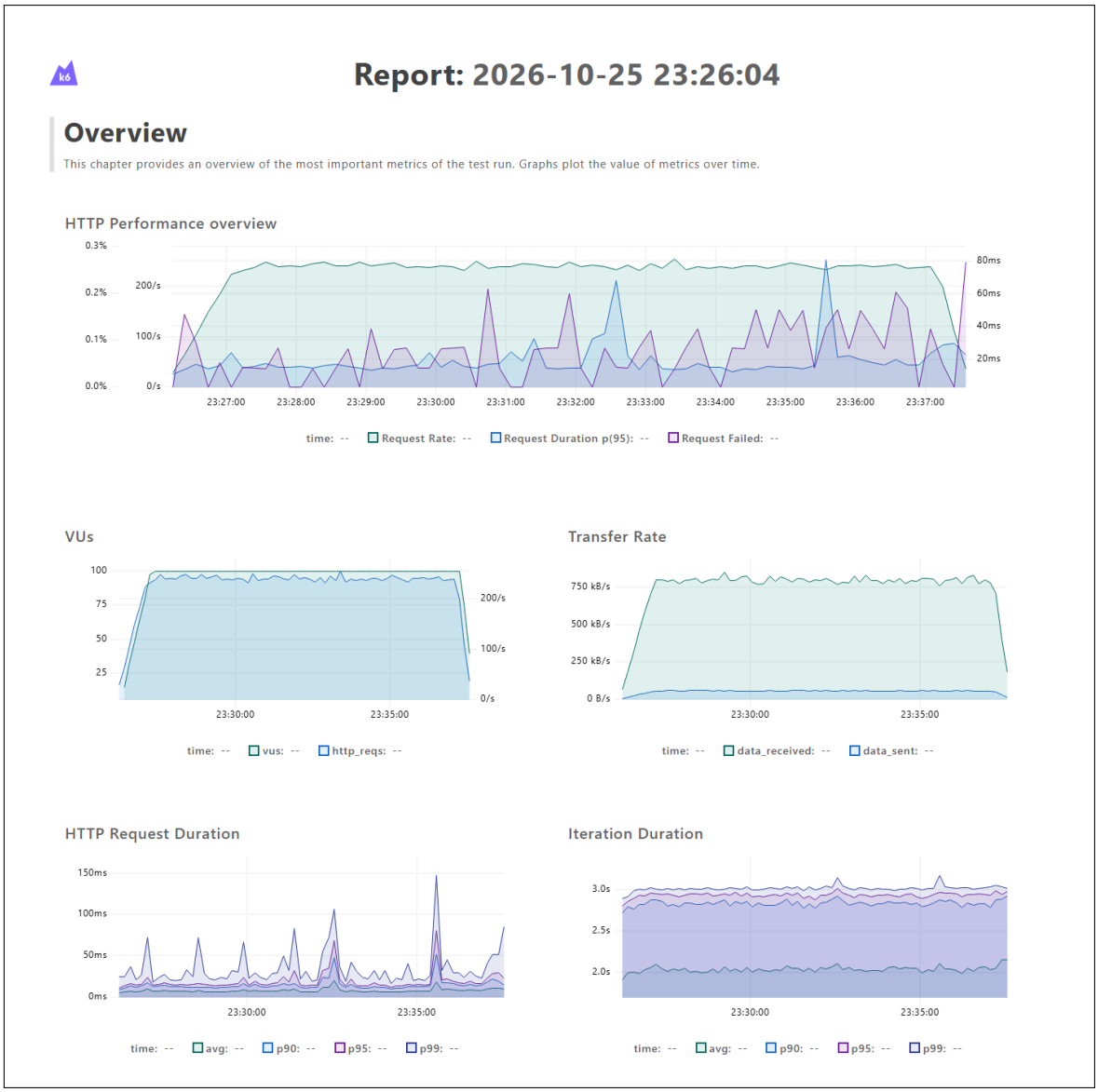


Figure 3: k6 dashboard for 100 users

Result: Pass.

4.3 Curfew Rush

This test only sends QR scans, like students arriving before the entry deadline. A real hostel of 1,000 students coming back over 30 minutes is less than 1 scan per second. We went up to 60 scans per second.

Every scan worked. The median was 10 ms and the p95 was 21 ms. Scans made after the deadline also created attendance incidents for the wardens, so the full scan logic ran, not just a simple insert.

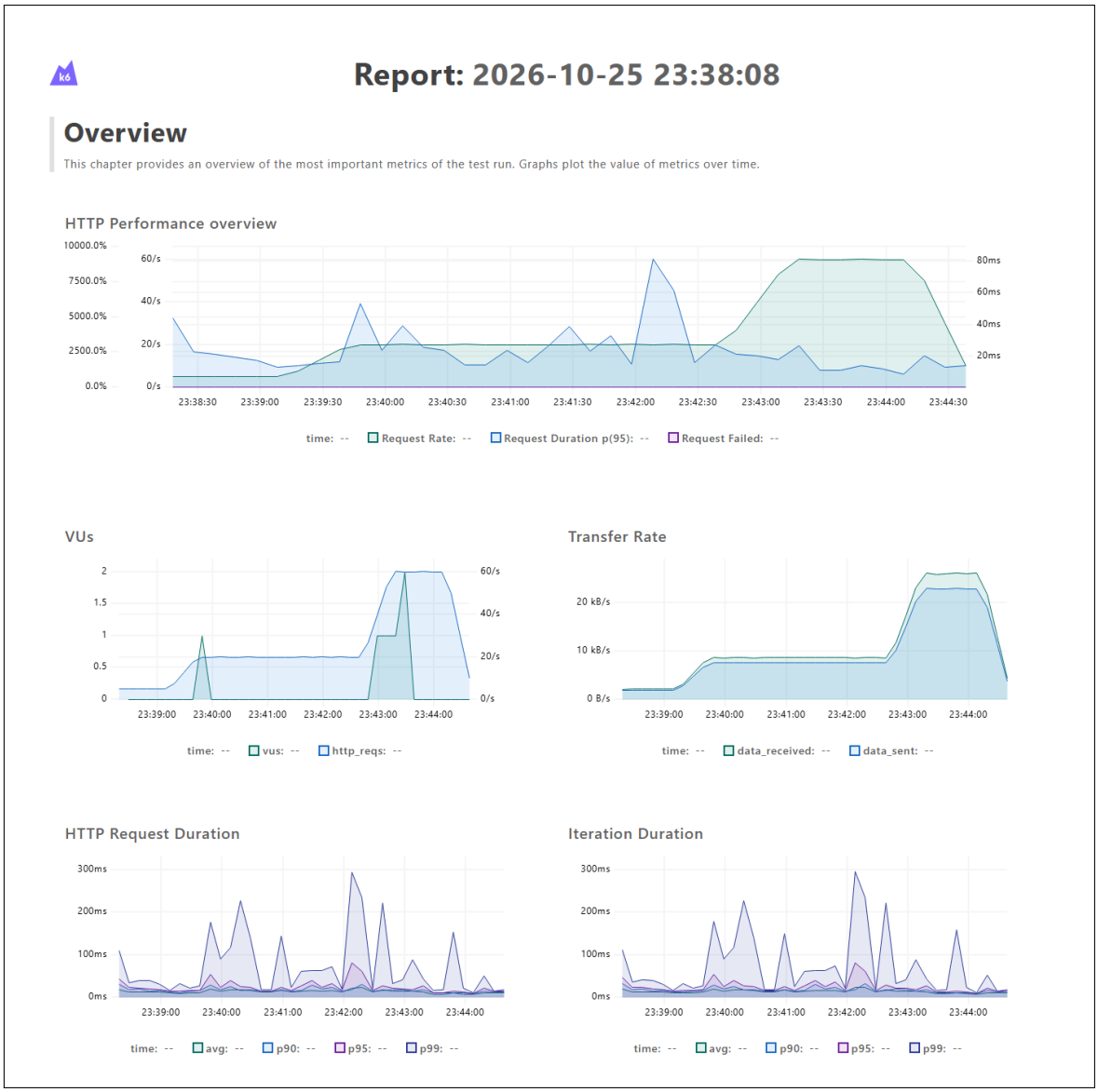


Figure 4: k6 dashboard for the curfew rush

Result: Pass.

4.4 Spike Test

The system went from 20 users to 300 users in 20 seconds. Response times went up for about a minute (p95 152 ms for the whole test) and went back to normal as soon as the spike ended.

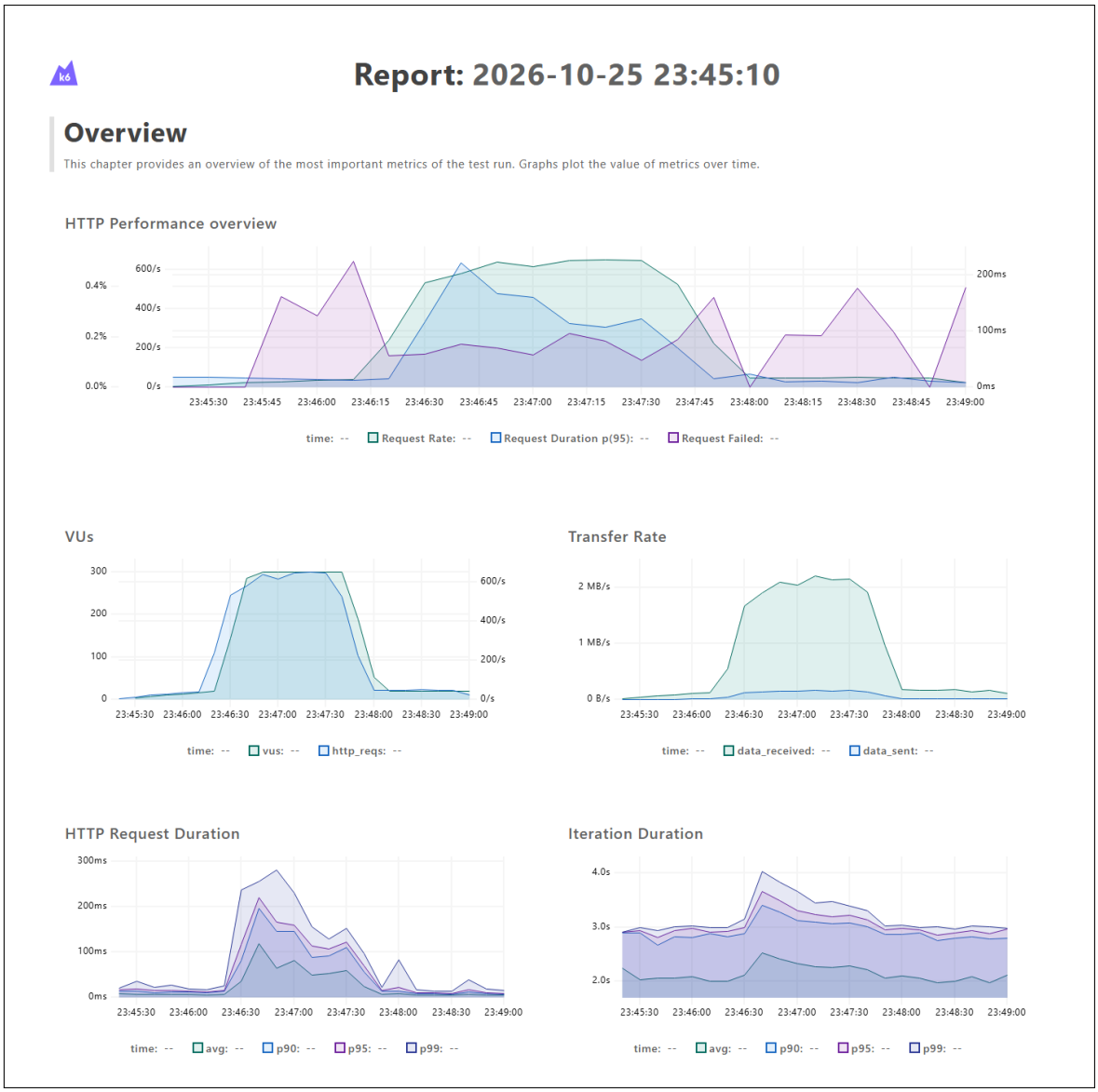


Figure 5: k6 dashboard for the spike test

Result: Pass.

4.5 Stress Test

Here users do not wait between actions, and traffic goes up every two minutes. Figure 6 and Figure 7 show what happened minute by minute.

```
Terminal

$ python analyse.py results/stress-points.json.gz -m
minute  req/s   median      p95      max  users
     1     23      5ms     13ms     75ms      1
     2     67      4ms     10ms     48ms      1
     3    104      4ms      9ms     59ms      2
     4    155      4ms     10ms     40ms      2
     5    202      5ms     11ms     33ms      2
     6    253      6ms     12ms     40ms      2
     7    299      6ms     13ms     52ms      3
     8    349      7ms     15ms    163ms     11
     9    402      8ms     28ms    152ms     32
    10    444      8ms     29ms    225ms     37
    11    502     13ms    103ms    293ms     68
    12    559     22ms    341ms    647ms    199
    13    618    106ms    596ms    862ms    365
    14    618    946ms   1176ms   1376ms    600
    15    264    964ms   1195ms   1376ms    600
```

Figure 6: Stress test minute by minute

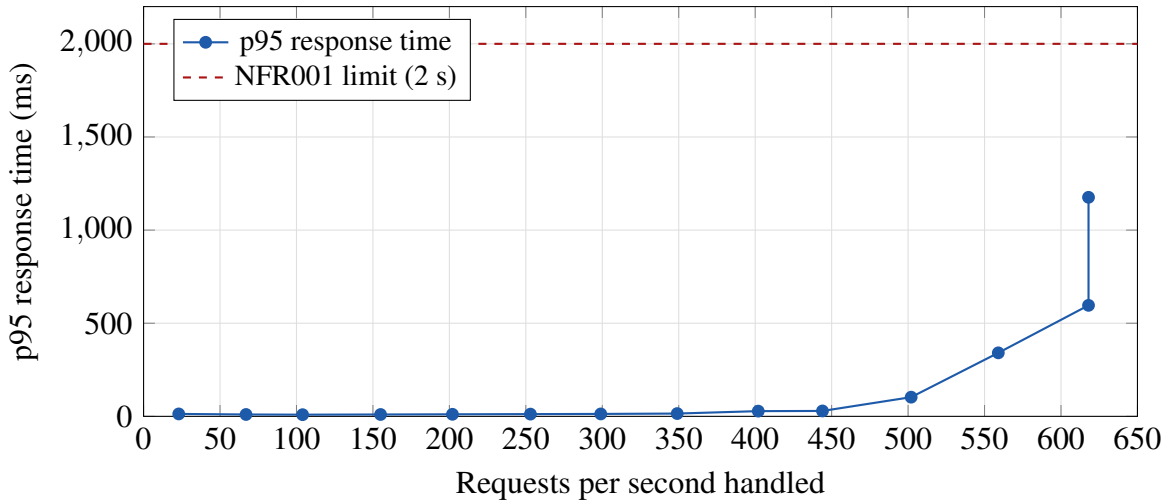


Figure 7: Response time as traffic goes up in the stress test

- Up to about 450 requests per second everything stayed fast, with a p95 under 30 ms.
- From about 500 requests per second response times started to go up.
- The server reached its top speed at about 620 requests per second.

So the backend on one CPU core handles roughly 450 requests per second before users notice anything. That is about twice what 100 users need, and even at the top the p95 stayed under 2 seconds.

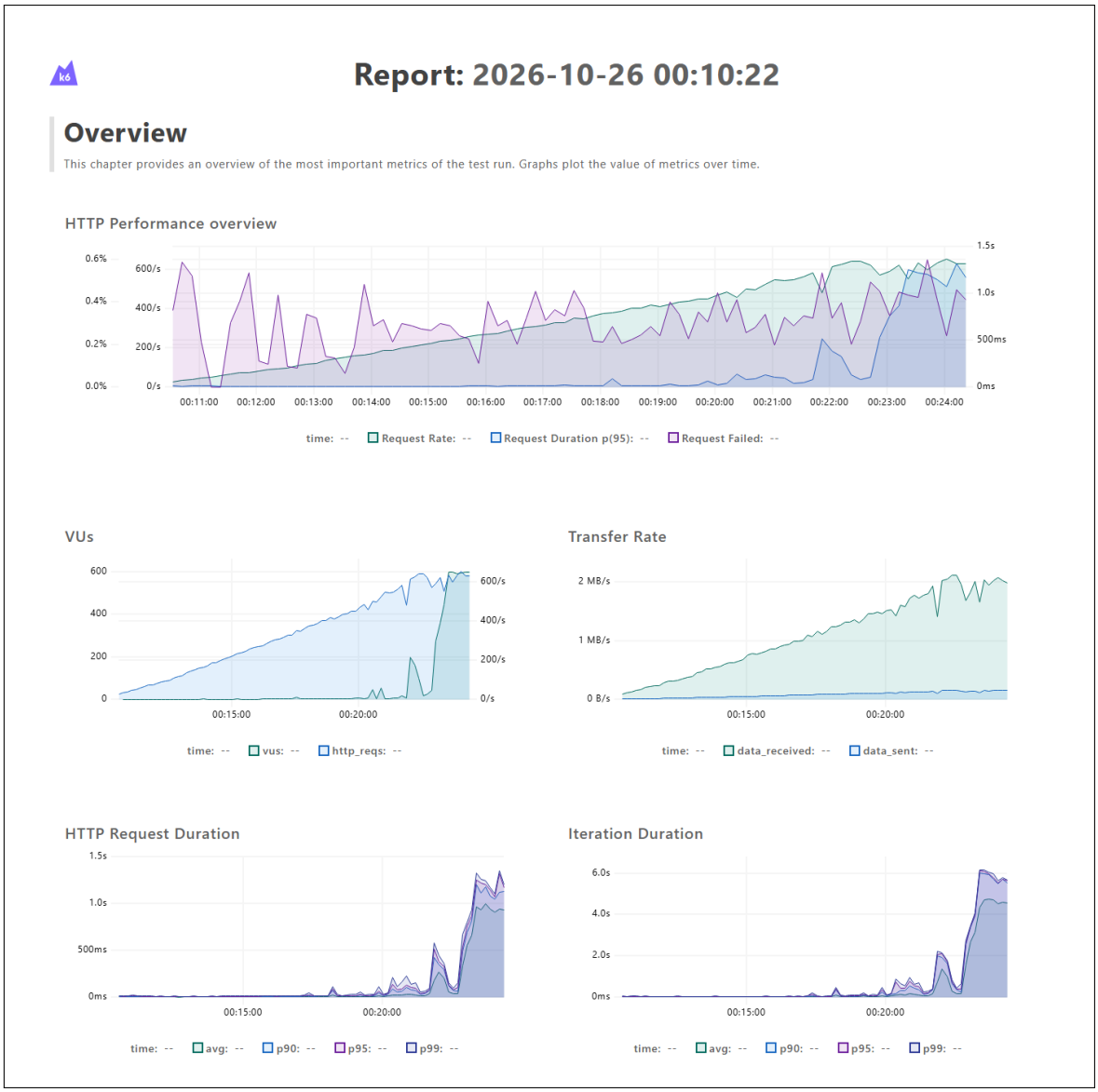


Figure 8: k6 dashboard for the stress test

Result: Pass.

4.6 Reports on Five Years of Data (NFR010)

Five wardens opened reports and exports at the same time for 4 minutes, all on the full 5 years of data.

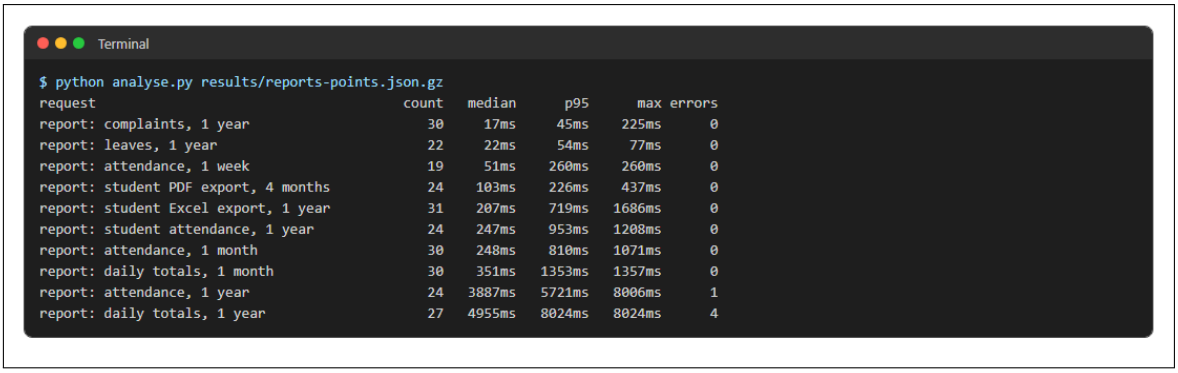


Figure 9: Time for each report type with 5 wardens

- Complaint and leave reports, single student reports and the student PDF and Excel exports all came back in well under 2 seconds.
- The one month attendance report and the one month daily totals also stayed under 2 seconds.
- Whole hostel attendance reports for a full year cover around half a million scans, so they take a few seconds. Wardens usually run these now and then, not every day.

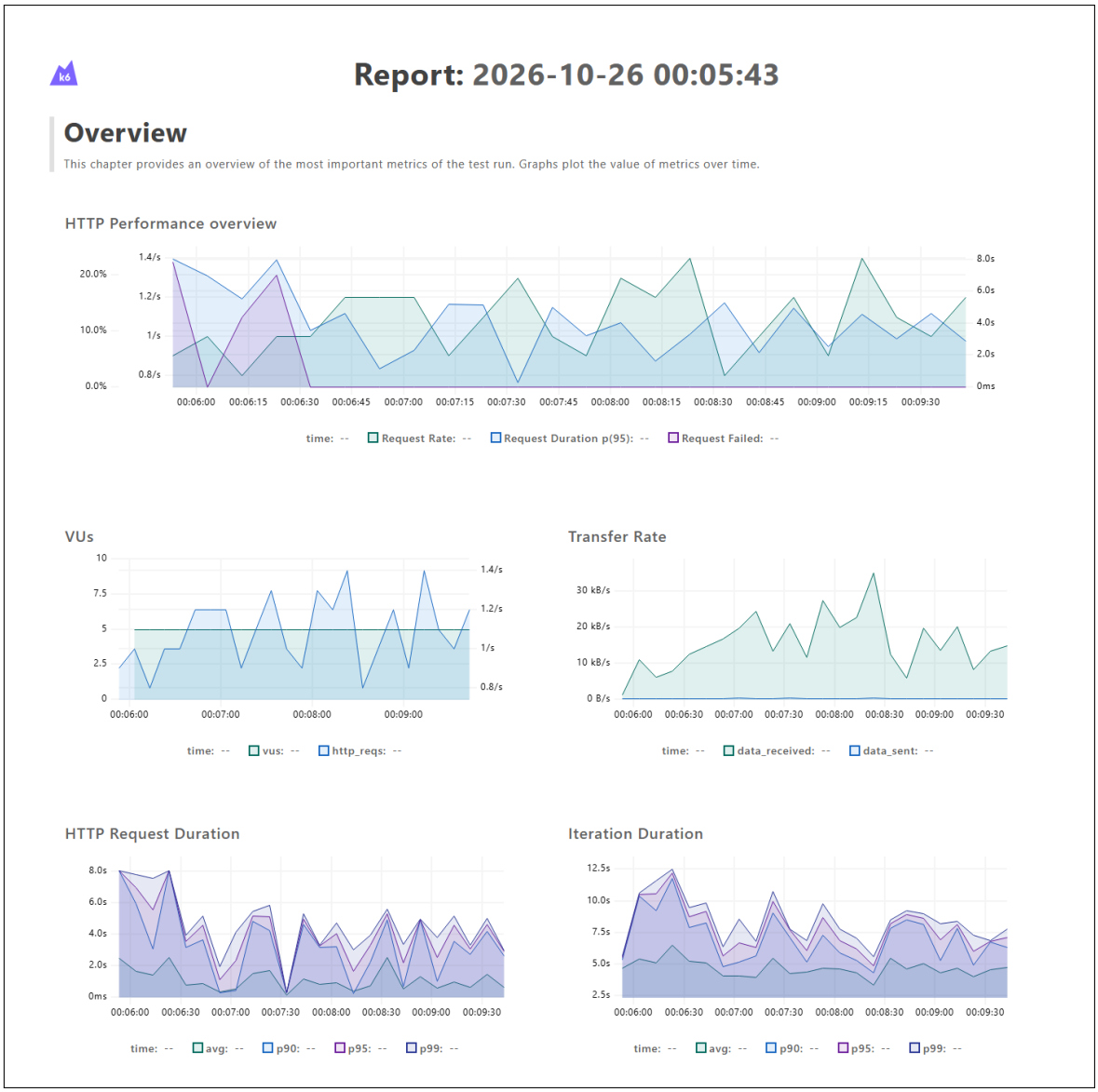


Figure 10: k6 dashboard for the reports test

4.7 Server Resources

We checked how much CPU and memory the two containers used every 5 seconds.

Test	Backend CPU (peak)	Backend memory (peak)	Database CPU (peak)
Load	47%	24 MB	2.7 cores
100 users	73%	27 MB	3.7 cores
Curfew rush	28%	26 MB	0.6 cores
Spike	105%	35 MB	3.8 cores
Stress	106%	57 MB	3.9 cores
Reports	2%	45 MB	2.3 cores

Table 11: Peak CPU and Memory (100% means one full core)

The Go backend uses very little memory, under 60 MB even at its top speed. Most of the work is done by the database.

CHAPTER 5

LIVE API ON RENDER

The live API runs at `https://hostelhive-backend-dev.onrender.com`. We gave it a light check: 5 users for 1 minute, calling the health check, the readiness check (which talks to the database), and a protected route without a token (which runs our login check and returns 401).

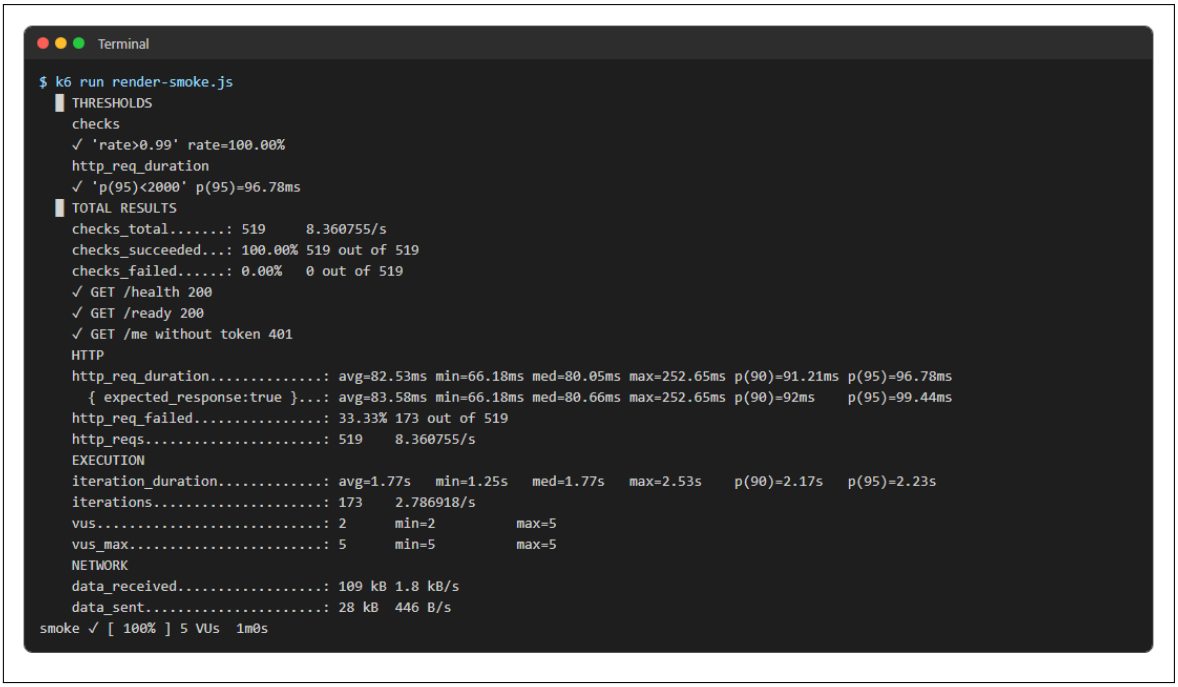


Figure 11: k6 result for the light check on Render

Request	Count	Median	p95	Max
`GET /health`	173	81 ms	106 ms	191 ms
`GET /ready`	173	81 ms	98 ms	253 ms
`GET /api/v1/me` without a token	173	79 ms	94 ms	135 ms

Table 12: Render Response Times

These times include the trip over the internet from Sri Lanka to the server. The k6 summary shows a 33% "failed" rate. That is only the 401 answers we asked for on purpose, because k6 counts every 4xx answer as failed. All checks passed.

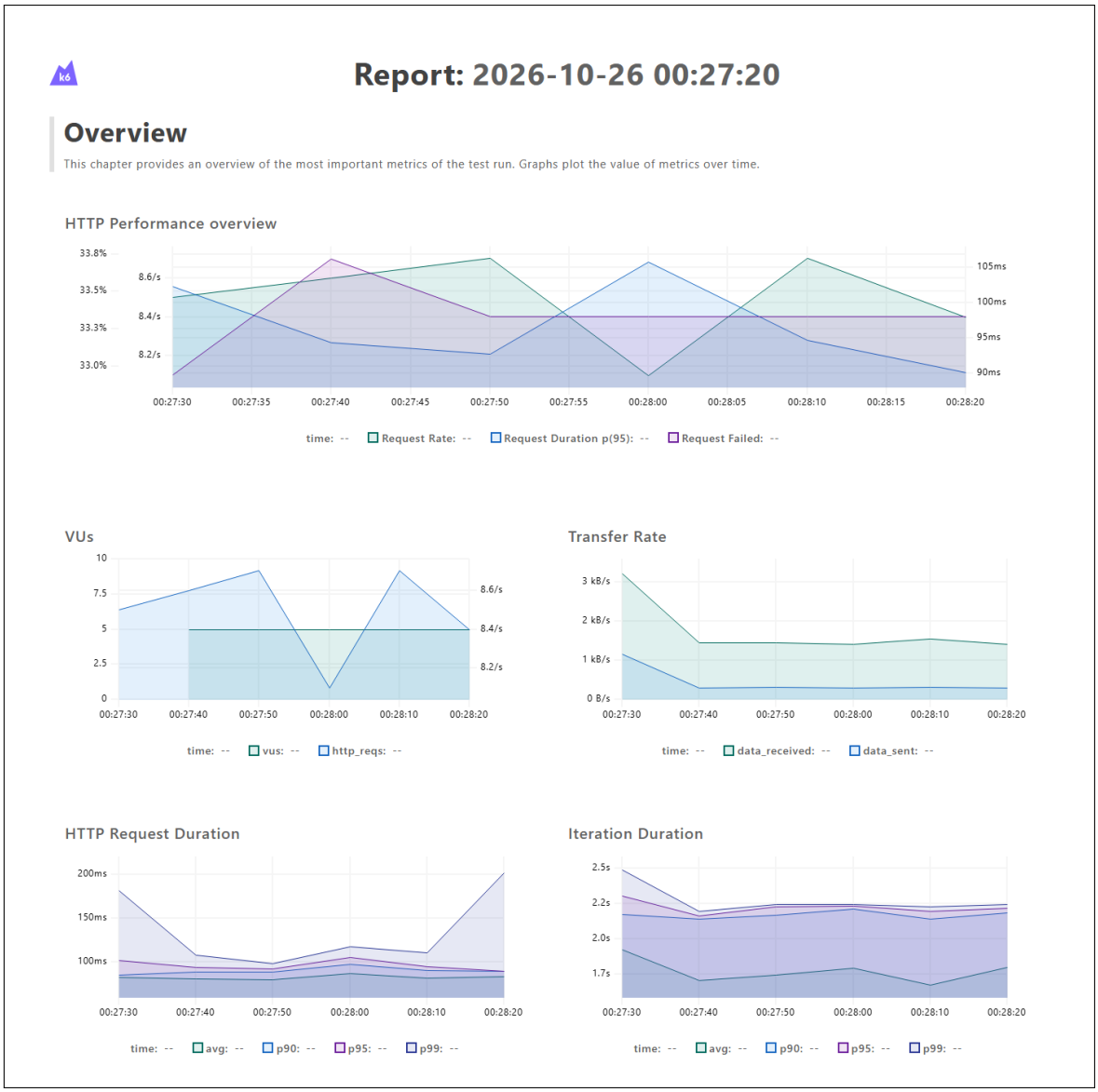


Figure 12: k6 dashboard for the Render check

On the plan we use, Render puts the server to sleep when nobody has used it for a while. The first request after that waits while it wakes up, and every request after it is fast.

CHAPTER 6

WEB DASHBOARD RESULTS

We ran Lighthouse on the live web dashboard at `https://web-kohl-delta-90.vercel.app`. The home page opens the student login, so we tested the student login and the staff login. All other pages need a login. Mobile runs use Lighthouse's normal phone settings: a slow 4G network and a phone CPU about four times slower than a laptop.

Page	FCP	LCP	TBT	CLS	Speed Index	Page size
Student login, desktop	0.4 s	0.5 s	0 ms	0.014	0.5 s	299 KB
Staff login, desktop	0.4 s	0.5 s	0 ms	0.014	0.5 s	299 KB
Student login, mobile	2.1 s	3.6 s	100 ms	0.023	3.5 s	433 KB
Staff login, mobile	2.2 s	3.5 s	40 ms	0.023	2.2 s	434 KB

Table 13: Web Dashboard Load Times

- On desktop both pages score 100 for performance. The page is ready in half a second.
- On a slow phone network the pages score 86 and 88, and the main part shows in about 3.5 seconds.
- Accessibility is 96 and best practices is 100 on every page.
- The page hardly moves while loading (CLS under 0.03).

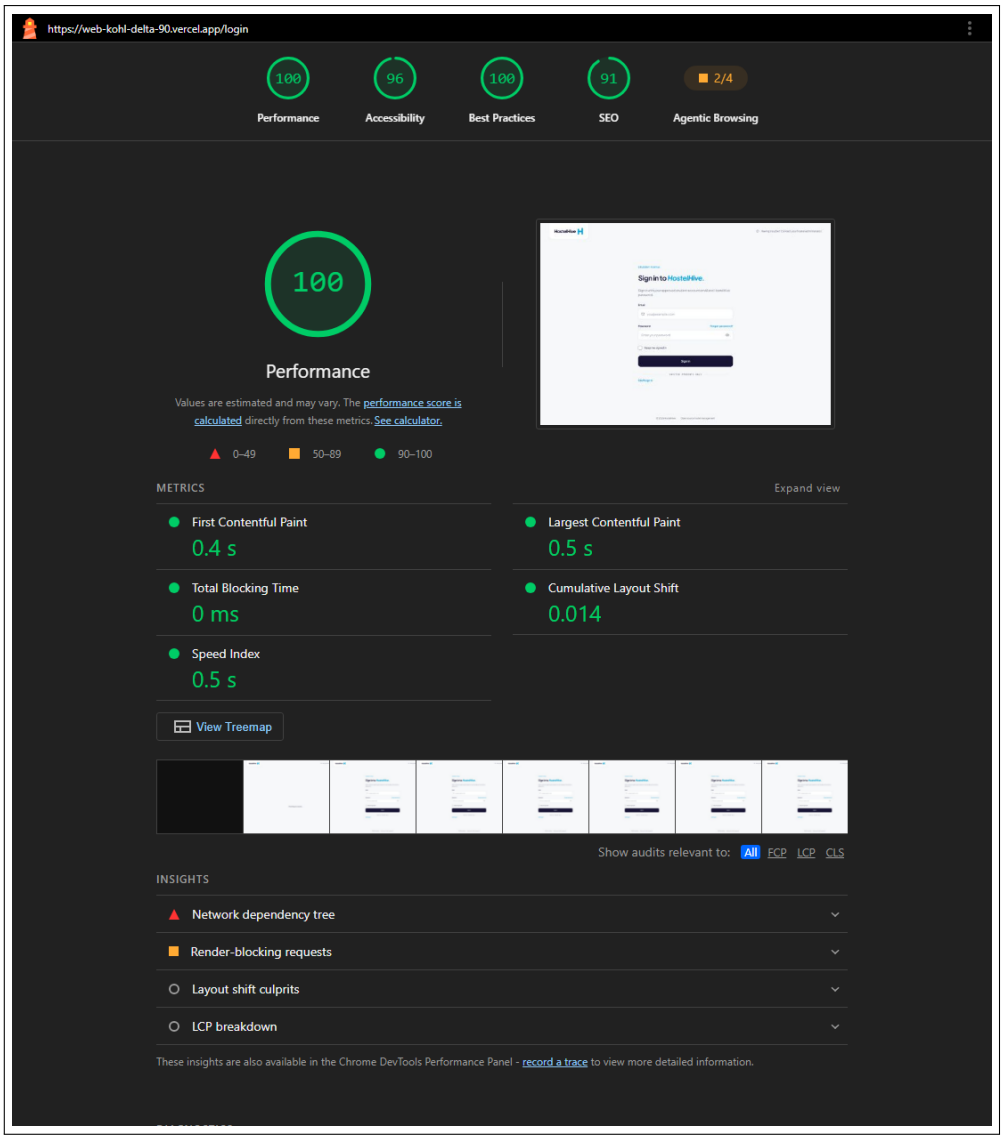


Figure 13: Lighthouse, student login on desktop

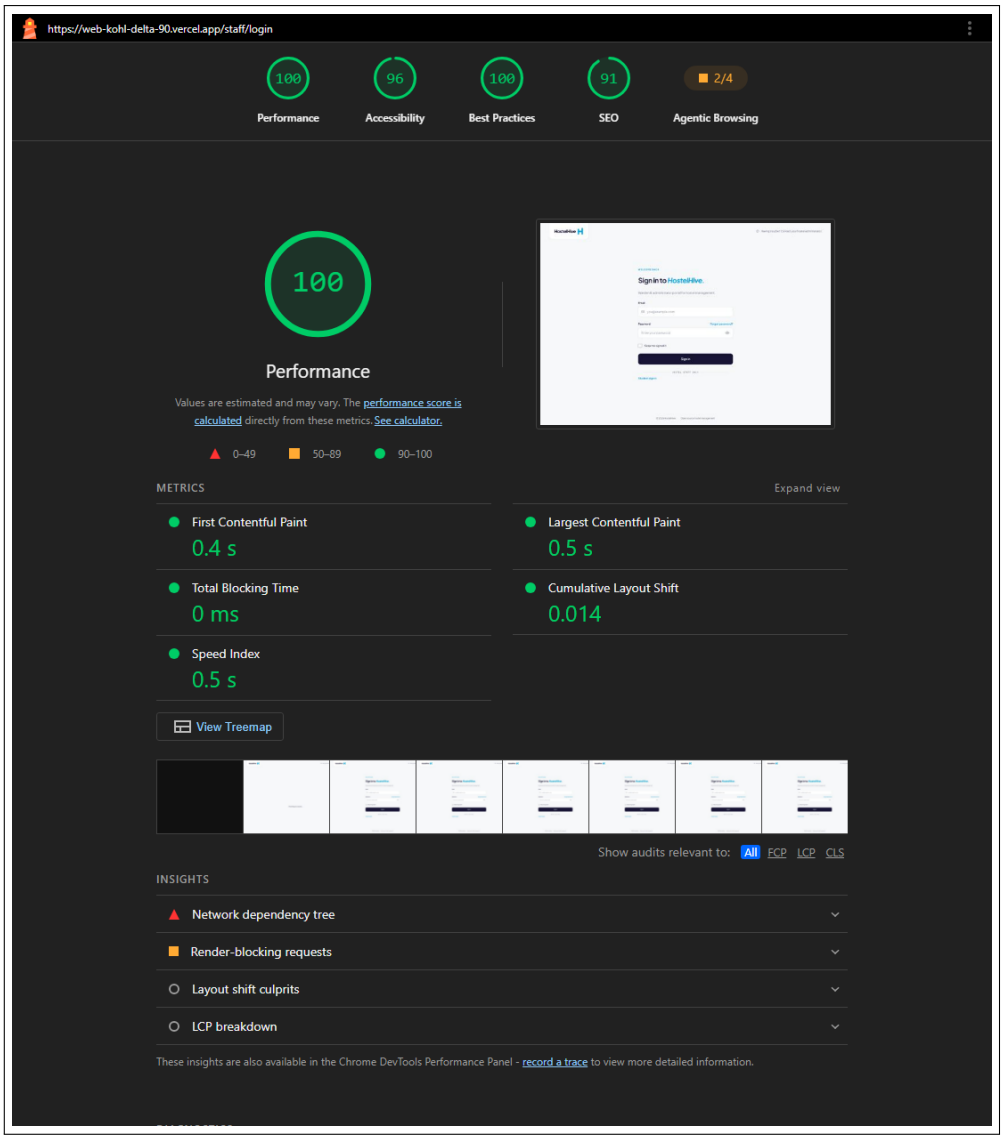


Figure 14: Lighthouse, staff login on desktop

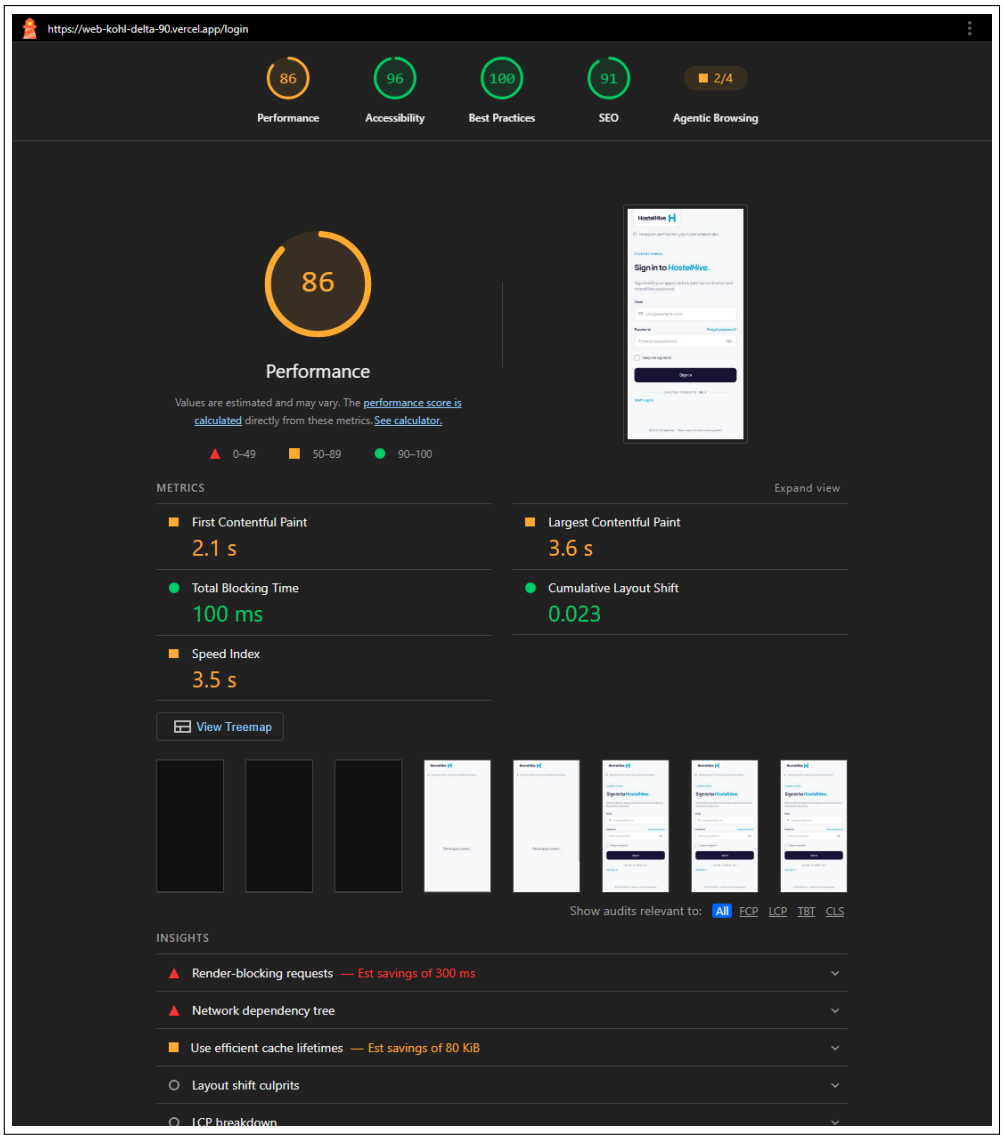


Figure 15: Lighthouse, student login on mobile

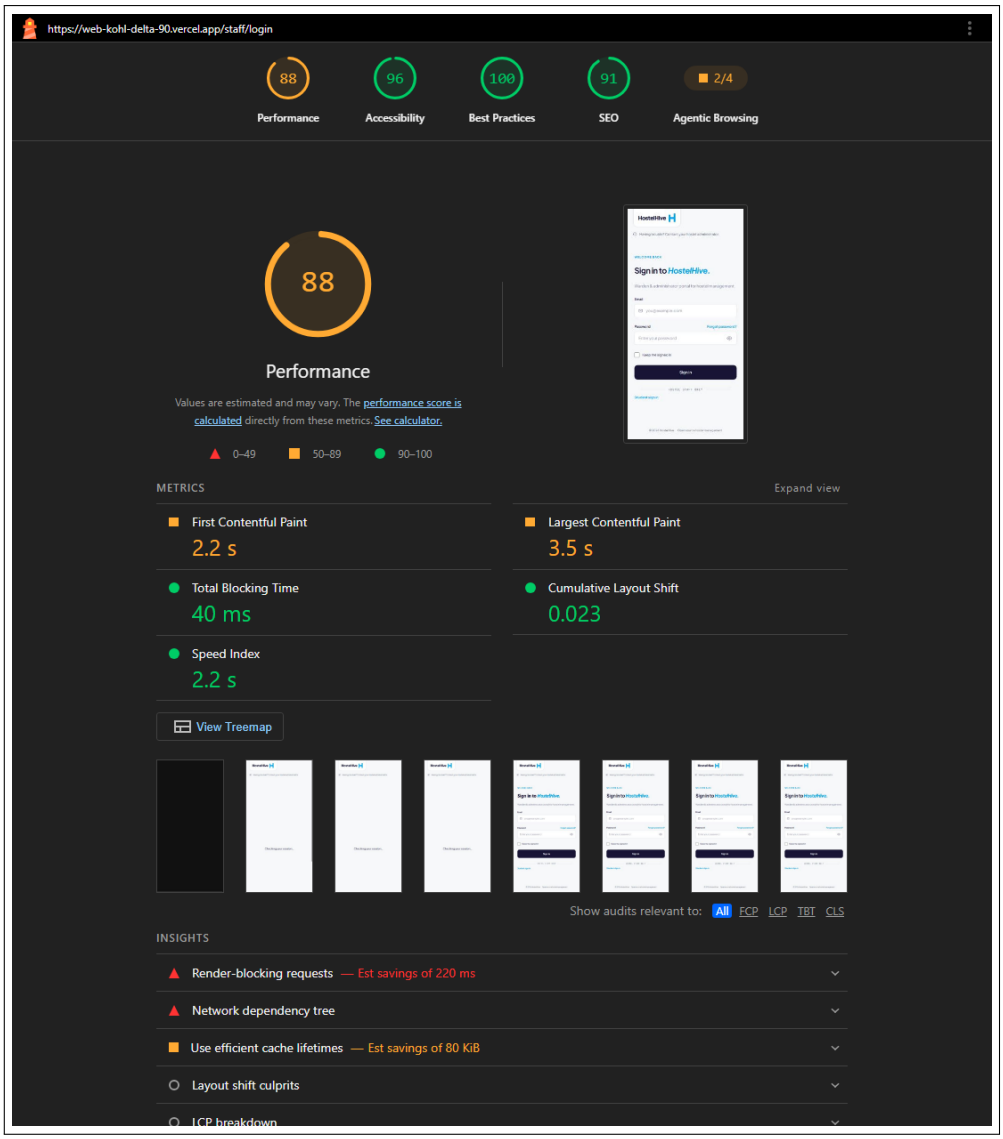


Figure 16: Lighthouse, staff login on mobile

CHAPTER 7

CONCLUSION

HostelHive meets the performance requirements in the SRS. Under normal load 95% of requests finished in about 20 ms, far inside the 2 second limit (NFR001). With 100 users at the same time there was no slow down (NFR002). Five years of attendance data were stored without archiving and every test ran on it (NFR010).

The backend also handled a sudden jump to 300 users, a gate rush far busier than a real curfew, and about 450 requests per second before response times started to rise. The web dashboard scores 100 on desktop and 86 to 88 on a slow phone network.

CHAPTER 8

REFERENCES

[1] Grafana Labs, *k6 documentation*, https://grafana.com/docs/k6/

[2] Google, *Lighthouse performance scoring*, https://developer.chrome.com/docs/lighthouse/performance/performance-scoring

[3] Render, *Free instance types*, https://render.com/docs/free

[4] Software Engineering Teaching Unit, University of Kelaniya, *SENG 34213 System Development Project Guideline*, Section 10.1.

[5] HostelHive Team, *Hostel Hive Software Requirement Specification*, Version 1.0, 2026.